Machine Learning

→ Why ML exists → Many problems we can solve with rule based codes but when data is more and complex, the rules become impossible to maintain.

→ AI vs ML vs DS

→ Types of ML

→ Data, features, Data splitting. → ML, helps computers to observe data, learn patterns & improve automatically.

→ Example, How computers understands can be compared as ~~A child~~

What is a dog? → This is the goal

Train:
- → If it is 4 legs
- → A tail
- → fur

Show examples

ID	~~This~~ Actual output
1)	This is a dog
2)	This is a dog
3)	This is NOT a dog

Train

AI: AI is the broader field of making machines perform tasks that normally require Human Intelligence.

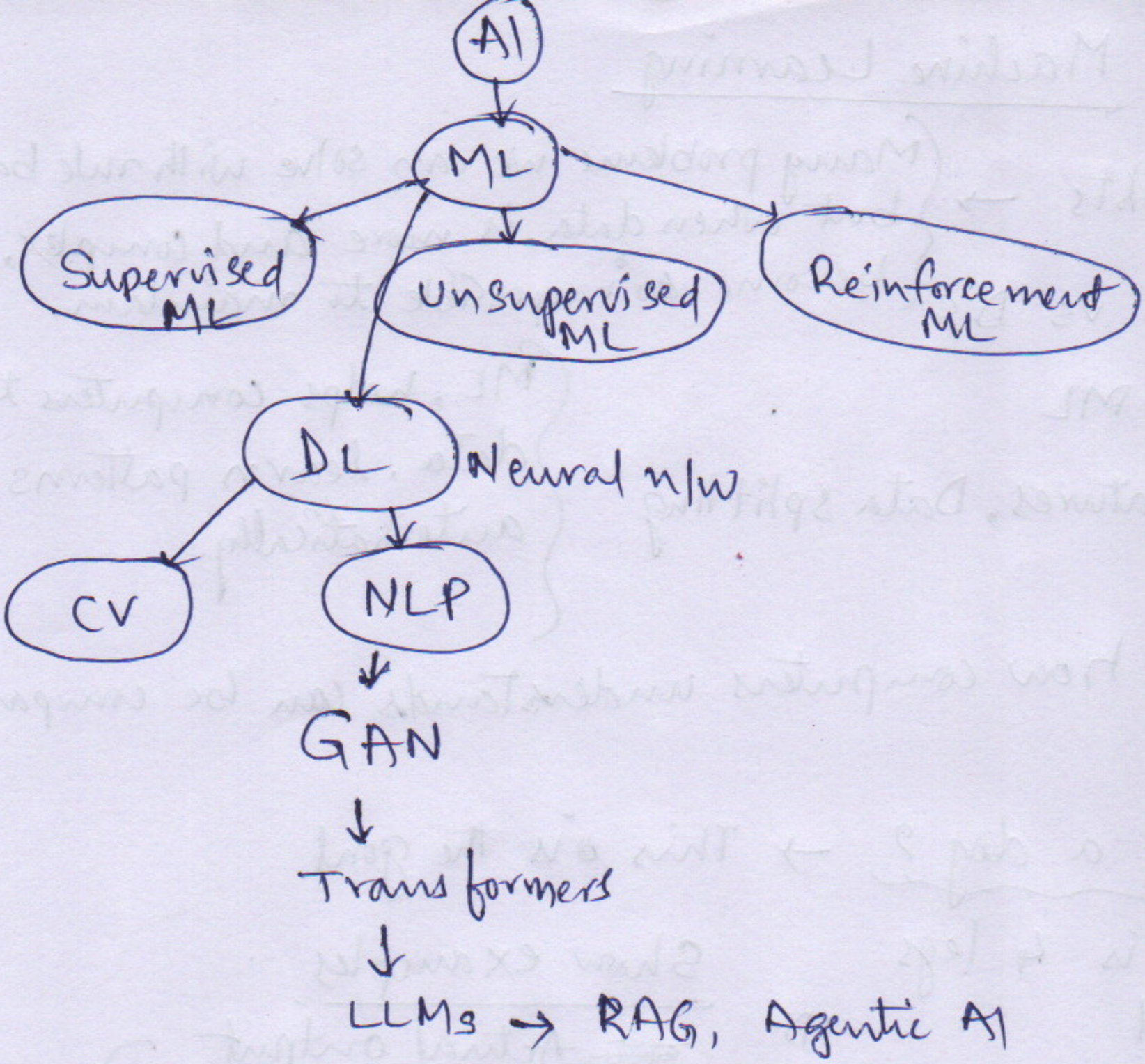
What is not an AI: AI does not mean emotion or self-awareness

RL → Driving

Agentic AI → Decision making

NLP → Understanding Language, translate

CV → Recognize faces



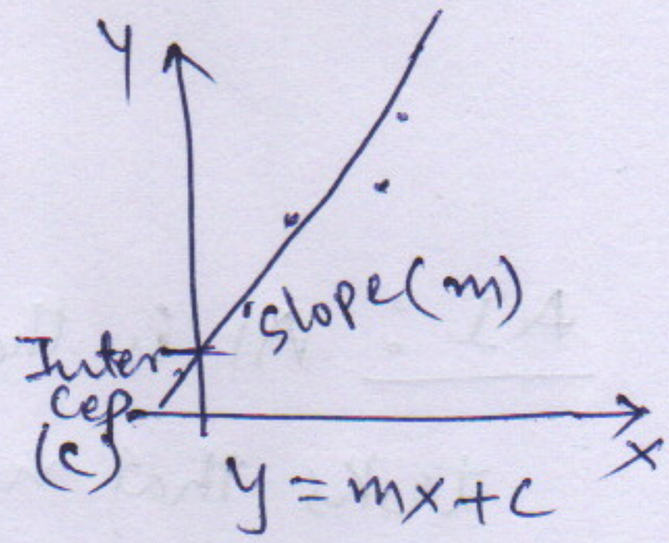
Supervised ML :

Experience(X)	Salary(Y)
1	30K
2	40 K
35	75K

Pattern : R/n b/n experience & salary

ML :

$y = mx + c$



Supervised ML, works on labelled data where feature(X) as input and Target(Y) as output in the ML

Regress Models : output is continuous [Salary, temp, stock price]

classification Models : output is categorical [Spam/not spam, cat/dog, fraud/not_fraud]

Identify Supervised/classification

Merits :
predict age, predict gender predict salary, predict pass/fail

Unsupervised ML

Examples: customer segmentation, anomaly detection recommendation grouping.

Reinforcement ML:

→ Rewards, punishment, trial & error

Dog Training

Rewarded → Correct action

Punishment → Wrong action

Assessment | Discussion:

Teacher: 1) Conducted an exam, some students got 100% marks in theory but they were not able do practicals on the given questions.

Hint: Student was memorised answers, DO WE ACCEPT THAT BEHAVIOUR (think in ML way)

2) Another student could secure 75% marks but could do all practicals accurately.

> Q: which student pattern you accept ? why ?

TRAIN – TEST – VALIDATION

Trains: Model development → ML(X, Y, Z)

Test: final testing ~~+ tune model + validation~~ (Unseen data)

Validation: model - testing + tuning (x', y', z')

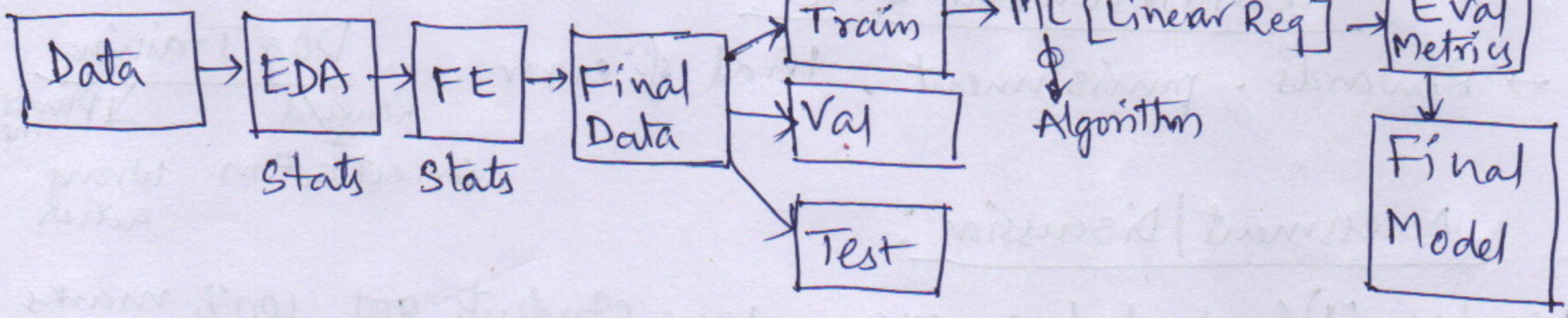
> Rediscuss teacher example with Final test

Data Split:

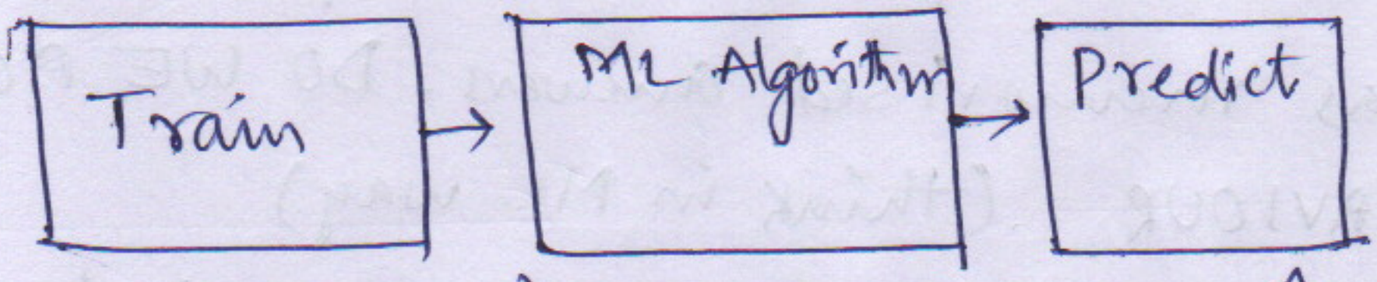
Train	Val	Test
90%	5%	5%
70%	15%	15%
80%	10%	10%

Brief: Evaluation, loss function, cost function, over/under/perfect fit, ideal model
Evaluation metrics [Regression, classification], linear, polynomial

Work flow :



Brief :



x_1	x_2	y (actual)
2	3	8
4	3	10
2	4	6
5	2	9

$y = mx + c$

$y = m_1x_1 + m_2x_2 + c$

$\hat{y}$ (predicted)

7

Evaluation metrics : wheather our model is learning or not ?

x_1	x_2	y	$\hat{y}$
2	3	8	9
4	3	10	8
2	4	6	7
5	2	9	7

$(y - \hat{y})$ error

-1 → Loss fun

+2

-1

+2

+2 Cost function

Loss fun: measures the error for a single data val

Cost funct: Sum of all loss func over the dataset

x_1	x_2	y	$\hat{y}$ (Underfitting)	$\hat{y}$ (Overfitting)	$\hat{y}$ (good fit)
2	3	8	2	8	8
4	3	10	14	10	11
2	4	6	2	6	8
5	2	9	8	2	8
6	2	7	2	8	6

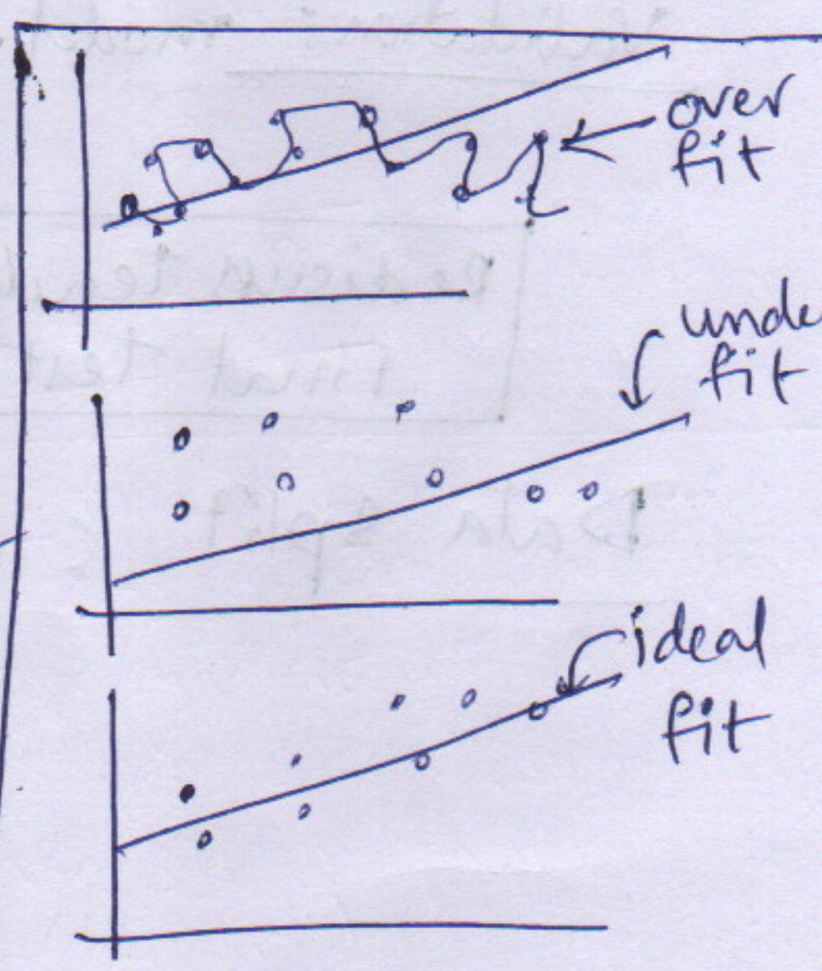
Train failed to learn

100% matches

ideal fit

Test failed

poor fit



Evaluation Metrics: Regression

(1) $\sum_{i=1}^{n} (y - \hat{y})$

(2) $\frac{1}{n} \sum_{i=1}^{n} (y - \hat{y})$

(3) MSE $= \frac{1}{2n} \sum_{i=1}^{n} (y - \hat{y})^2$ — modified MSE

Mean Square Error

(4) MAE $= \frac{1}{n} \sum_{i=1}^{n} |y - \hat{y}|$

Mean Absolute error

(5) RMSE $= \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y - \hat{y})^2}$

Root Mean Square error

Evaluation Metrics: classification

TP, TN, FP, FN (Confusion matrix)

Actuals (Y)		Spam (+ve)	Not Spam (-ve)
	Spam (+ve)	TP 600	FN 300
	Not Spam (-ve)	FP 100	TN 900
		Spam +ve	not-Sp -ve
		Predicted (Y)	

True Positive (TP): Truely classified as positive

True Negative (TN): Truely classified as negative

False Negative (FN): Falsely classified as Negative

False Positive (FP): Falsely classified as Positive

Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$ (all true cases)

Precision $= \frac{TP}{TP+FP}$ How often we are correct

$= \frac{\text{Actual positive}}{\text{Total predicted positive}}$

Considers +ve predictions inaccurately

Considers -ve prediction inaccuracy

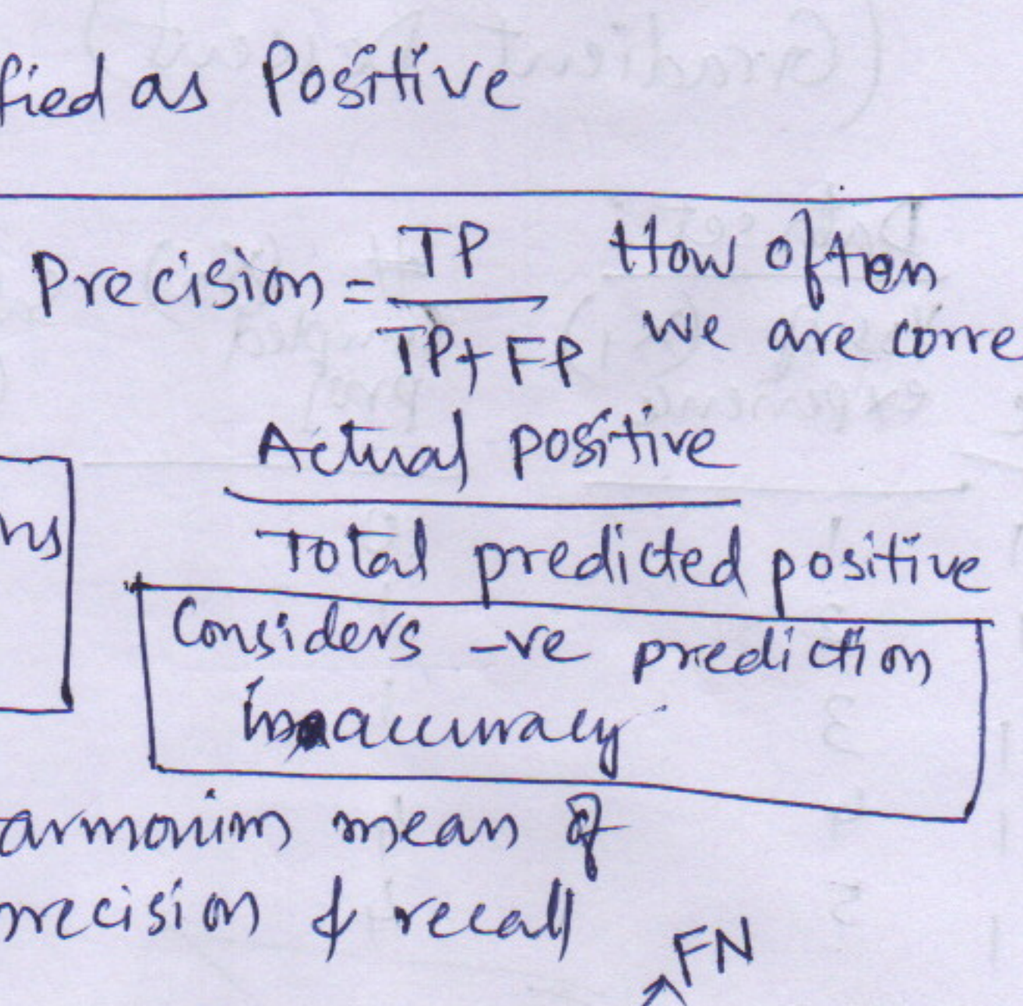
Recall $= \frac{TP}{TP+FN}$

$F1 = \frac{2(\text{precision} \times \text{Recall})}{\text{precision} + \text{Recall}}$ Harmonic mean of precision & recall

Classification (computer vision):

Threshold > 70%

Data	ML	>70%	Y
1	20%	F	F
2	75%	T	T
3	25%	F	F
4	80%	T	T
5	57%	F	T → error



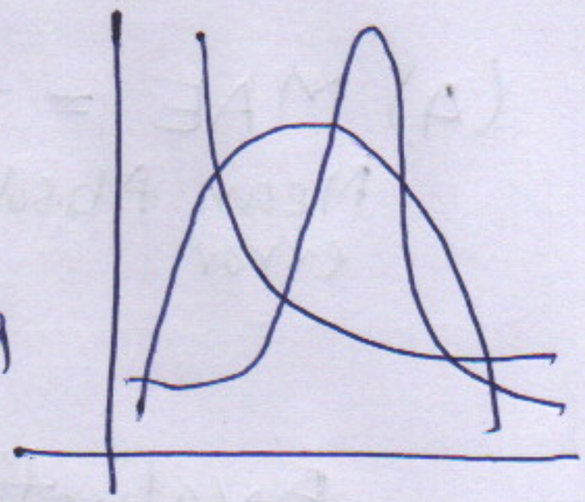
Polynomial equation: The exponential value is ≥ 2 (x^2, x^3, x^n)

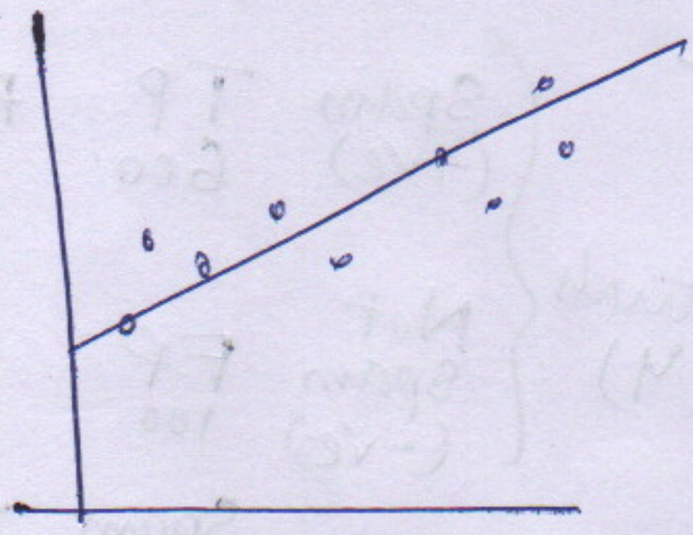
↳ ~~This graph is~~ usually the graph contains curves with peaks, valleys and U-shapes.

Quadratic eq: $y = x^2 + 3x + 2$

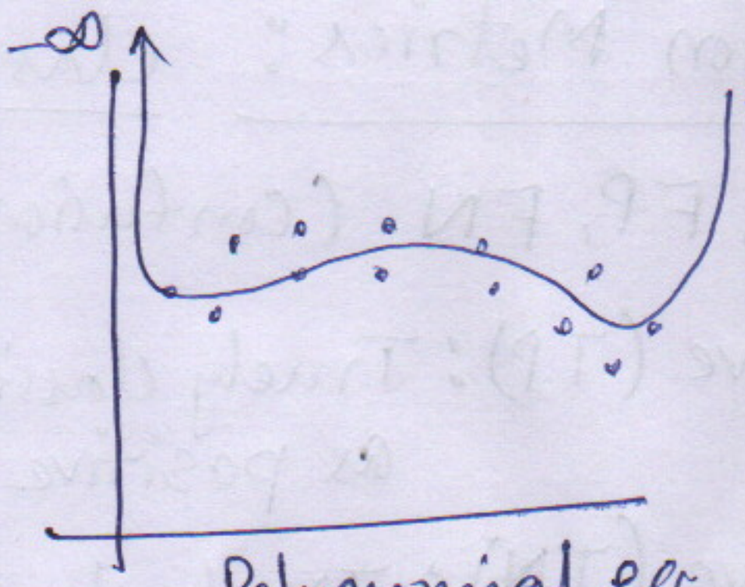
Cubic eq: $f(x) = x^3 - 4x^2 + x - 5$

Quadratic eq: $y = x^4 + 3x^3 + 5x^2 + 8x - 9$





Linear eq



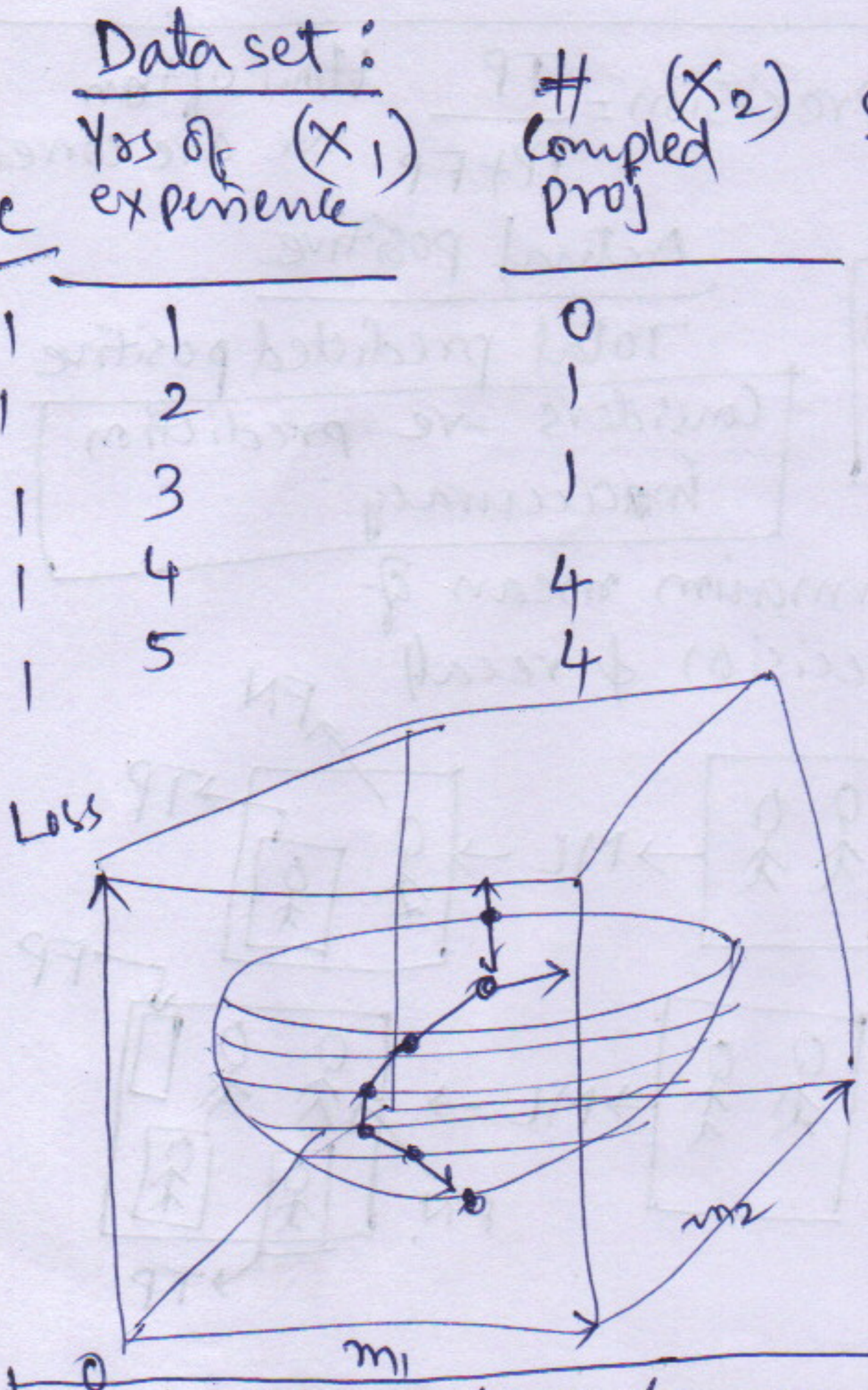
Polynomial eq

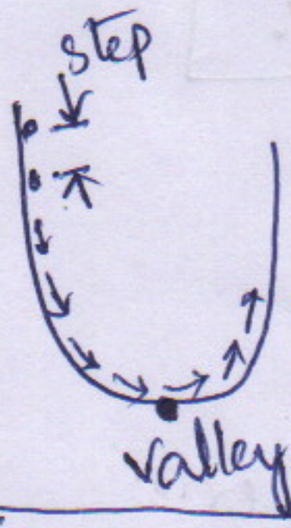
Optimization Algorithm (Gradient Descent):- Optimal value of m_1, m_2 to nudge the parameters to how much to move / step

Dataset:

c	Yrs of experience (X_1)	# Coupled proj (X_2)	Salary (K) (y)	$\hat{y}$ ($m_1x_1 + m_2x_2 + c$)	error ($y - \hat{y}$)	$\hat{y}$_new	error_new
				c=1, m_1=0, m_2=0		c=1, m_1=0.01, m_2=0.01	
1	1	0	40	1	39	1.01	
1	2	1	48	1	47	1.03	
1	3	1	55	1	54	1.04	
1	4	4	70	1	69	1.08	
1	5	4	80	1	79	1.09	

Cost function: 288





GD: Goal is to find min loss
→ Uses differentiation to cal slope
→ Slope/gradient will tell us direction of steepest ascent (uphill)
→ Take step in opp dir (downhill)

GD describes output changes for tiny change in input

GD: Discuss an example of motor cycle pedal, break for an optimum speed, direction etc

fn $\Rightarrow 3x^2+5$ $\Rightarrow$ ~~tiny~~ tiny nudge $\Rightarrow x = (x+h)$

$\Rightarrow 3(x+h)^2+5 \Rightarrow 3(x^2+h^2+2xh)+5$

$\Rightarrow 3x^2+3h^2+6xh+5$

h → small amount of

Find change: change = New output − Original output

$\Rightarrow (3x^2+3h^2+6xh+5) - (3x^2+5)$

$\Rightarrow 6xh+3h^2$

$$\text{Rate of change} = \frac{\text{change in output}}{\text{change in input}} = \frac{6xh+3h^2}{h}$$

$$\Rightarrow \frac{h(6x+3h)}{h} \Rightarrow 6x+3h$$

nudge(h) $\Rightarrow$ if $h=0$ then $6x$ $\Rightarrow$ which is the derivate of input

Rate of change in the input

Multiple Parameters: $3x^2+5xy+y^3$

→ We use partial derivatives instead of simple "

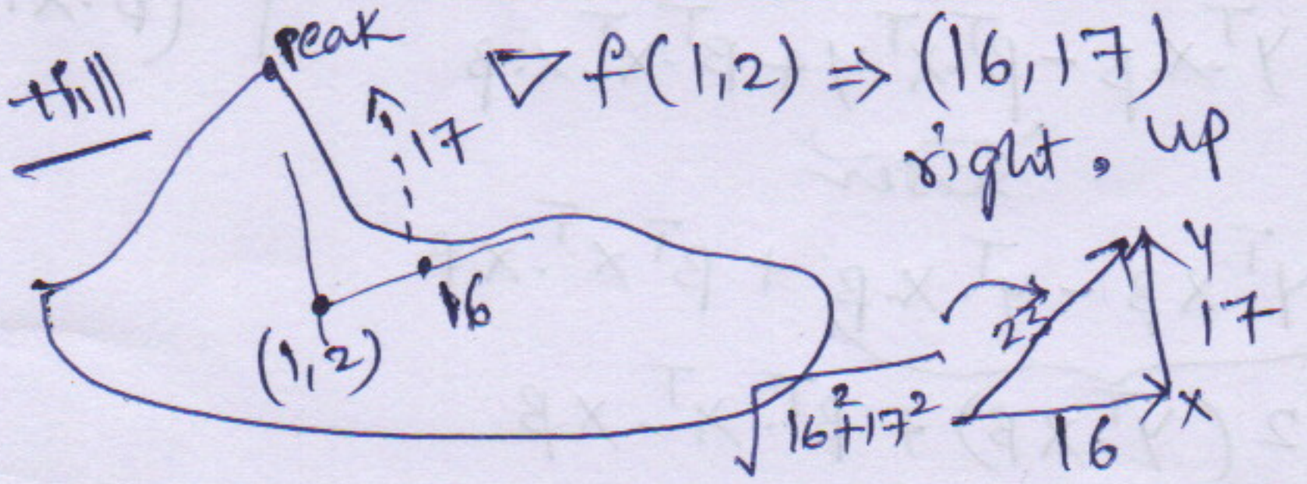
rate of change w.r.t $x \Rightarrow \frac{df}{dx}$ | $y \Rightarrow \frac{df}{dy}$

$x \Rightarrow 6x+5y$ | ~~$y \Rightarrow 5x+3y^2$~~ $y \Rightarrow 5x+3y^2$

$\nabla f(x,y) = (6x+5y, 5x+3y^2)$

$\nabla f(1,2) = (6+10, 5+12) \Rightarrow (16,17)$

Derivative gives you direction of steepest Ascent



decesion:

Always go in opposite direction to reach valley

Discussion: $BMI = \frac{weight}{height^2}$

Parameters → w, h

height → 1.8 m (constant)

$$\frac{df}{dw} = \frac{1}{h^2} = \frac{1}{(1.8)^2} = \approx 0.309$$

If ~~any~~ weight increases by 1 point ⇒ ~~BMI × 0.309 increases~~

~~BMI = BMI_old + (BMI × 0.309)~~

BMI = BMI_old + (weight change × 0.309)

New weight = 2kg : 2 × 0.309 = 0.618

Gradient Descent : ~~BMI~~ Goal is to reach minimum loss

→ Use differentiation to calculate slope of the cost fun at current position. Repeat until it reaches downhill.

Differentiation : for gradient descent

Integration : for back propagation

gradient $= \frac{1}{2m}(y-\hat{y})^2$ ⇒ modified MSE

J ⇒ Cost function β → parameter ($m_0 + m_1 + m_2$)

X → input (x_1, x_2)

$$J(\beta) = \frac{1}{2m}(y-\hat{y})^2$$

$$= \frac{1}{2m}(y - X.\beta)^2$$

$$= \frac{1}{2m}(y-X.\beta)^T(y-X.\beta)$$

Constant differentiation

$\Rightarrow (y^T - (X.\beta)^T).(y - X.\beta)$ → Rule-1

$\Rightarrow (y^T - (\beta^T.X^T)).(y - X.\beta)$ → Rule-2

$\Rightarrow y^T.y - y^T.X.\beta - \beta^T.X^T.y + \beta^T.X^T.X.\beta$

scaler

$\Rightarrow y^T.y - y^T.X\beta - y^T.X.\beta + \beta^T X^T.X\beta$

$\Rightarrow y^T.y - 2(y^T.X\beta) + \beta^T.X^T.X\beta$

matrix multiplication :—

$a^2 = a \times a$

$A^2 = A \times A^T$

Rules :

1) $(A-B)^T = A^T - B^T$

2) $(A.B)^T = B^T.A^T$

$B^T.A^T \neq A^T.B^T$

Scaler Rule : is al

equal to its own tra

$(\beta^T.X^T.y) = (\beta^T.X^T$

Take the gradient $\Rightarrow \frac{\partial S}{\partial \beta}$ $\Rightarrow 0 - 2(X^T \cdot Y)^T \cdot \beta$

$\Rightarrow -2X^T \cdot Y + 2X^T X\beta$

$\Rightarrow 2X^T \cdot X\beta - 2X^T Y$

$\Rightarrow 2X^T (X\beta - Y)$

$\Rightarrow 2X^T (\hat{Y} - Y)$

$\Rightarrow Y^T \cdot Y - 2(Y^T \cdot X\beta) + \beta^T \cdot X^T \cdot X\beta$

$\Rightarrow (X^T \cdot X + (X^T X)^T)\beta$

$\Rightarrow (X^T \cdot X + X^T \cdot X)\beta$

$\Rightarrow 2X^T \cdot X\beta$

$J(\beta) = \frac{1}{2m} \cdot (2X^T (\hat{Y} - Y))$

Gradient of Cost func $= \frac{X^T (\hat{Y} - Y)}{m}$

$\Rightarrow X.T @ (Y_{pred} - Y) / m$

i.e { direction & rate of change $\times$ 0.01 (learning rate to control the step)
direction: towards hill; (Ypred - Y)
rate of change: step towards hill but we keep it low with learning rate

gradient × step

direction + step

down ← m1 − gradient → top

m_1

$m_{1_new} = m_1 - gradient$

Data:

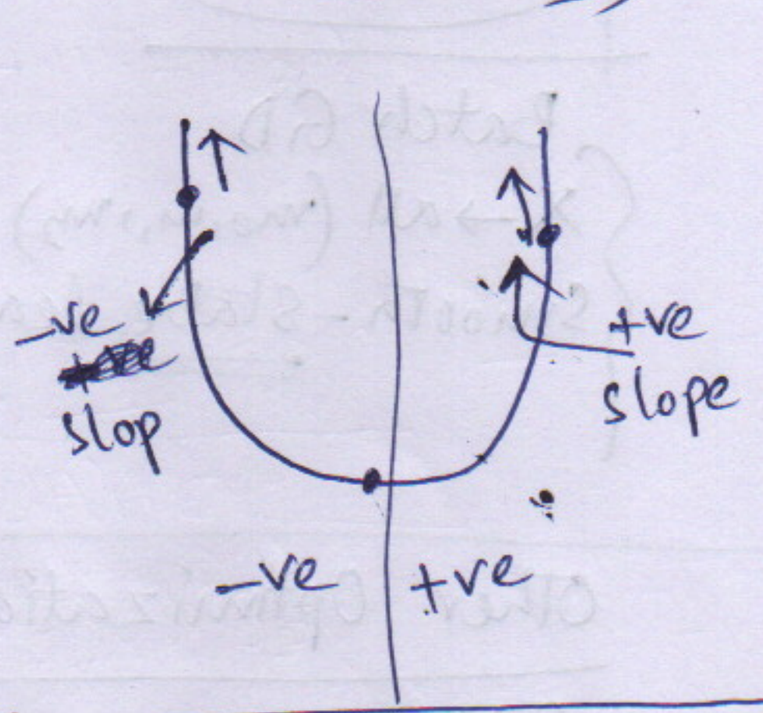
```
X = np.array([[1,1,0],[1,2,1],[1,3,1],[1,4,2],[1,5,3]])
Y = np.array([39,47,56,68,77])
beta = np.zeros(3)   # b0, b1, b2
learning_rate = 0.01
Y_pred = X @ beta.T   # @ dot product
m = len(Y)
errors = Y - Y_pred
gradient = (X.T @ (Y_pred - Y))/m
```

Loop-2

```
beta = beta - learning_rate * gradient  #new beta (m0, m1, m2)
Y_pred = X @ beta   # Transpose already done so no need to do again
errors = Y - Y_pred
```

Loop-n

```
beta = beta - learning_rate * gradient  # final m0, m1, m2
errors = Y - Y_pred   # errors should be very small
```



Discussion: Tortoise & Hair example

$m_0 = 22.17$
$m_1 = 14.68$
$m_2 = 6.24$

prediction = $m_0x_1 + m_1x_1 + m_2x_2$ # Linear Regression

$m_0 = 22.17$

$m_1 = 14.68$

$m_2 = -6.24$

Salary_pred = $1 * m_0$ + years_of_exp $* m_1$ + Completed_proj $* m_2$

Work flow : Linear Regression

→ Initialize weights to 0 → Y_pred = X.β

→ $\beta_{new} = \beta_{old} - \frac{\partial Loss}{\partial \beta} \times$ learning rate

7/6

Divide data into Batches to fit in RAM

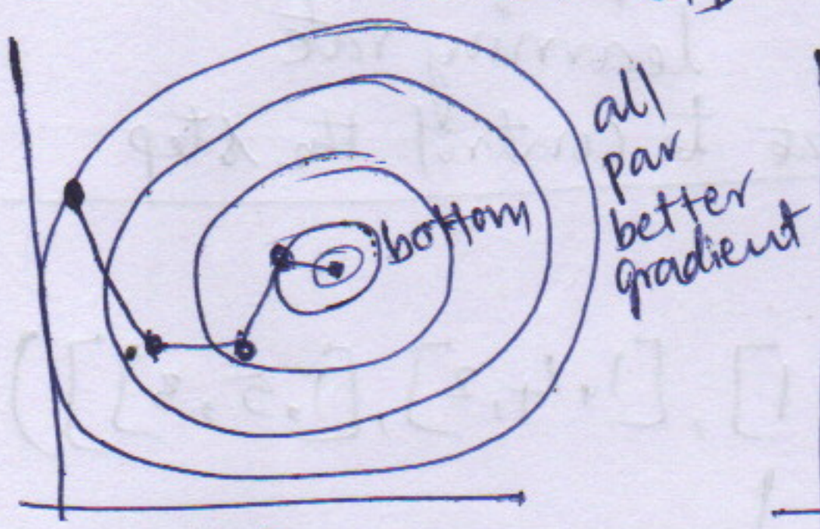
Sochastic GD ⇒ Batch size = 1

Mini batch GD ⇒ Batch size > 1 & ~~less~~ less than total count

Batch GD ⇒ Batch size = all data

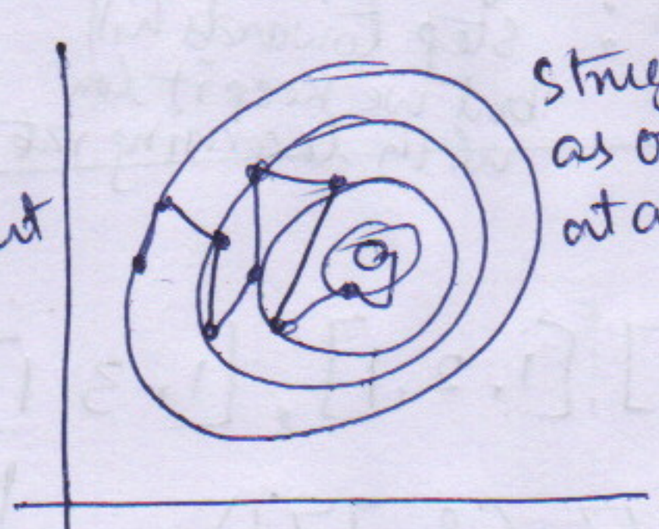
(m_0, m_1, m_2)

↳ each exe. once at a time



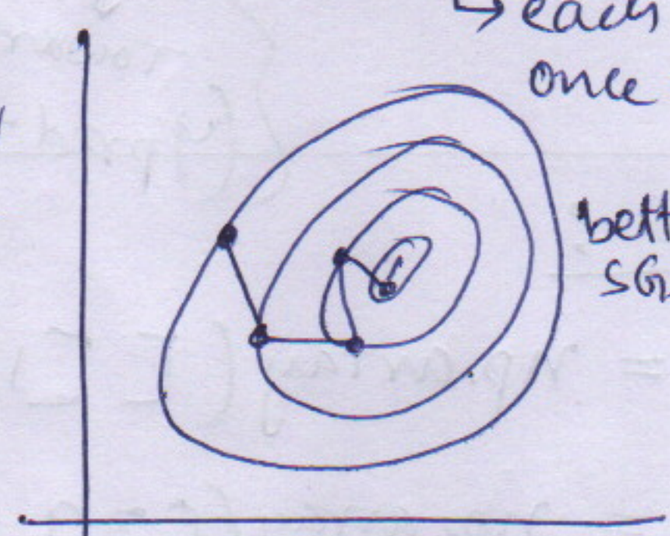
Batch GD

- X → all (m_0, m_1, m_2)
- Smooth - Stable learning



Sochastic GD

- for i in X : weight update
- → Uses less memory
- → global solution
- → very fast



miniGD

- → perfect middle ground
- → Combines both SGD & Batch GD
  - SGD → fast
  - Batch GD → Stable

Other Optimization Algorithms :-

→ RMS prop → Adam

→ Adagrad → GD

Linear Regression

GD

- → parameters updates in iteration
- best used for
- → medium_large data set
- → heavy computation
- → $J(\beta) = \frac{1}{2m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$

OLS (Ordinary Least Squares)

- → finds optimal param in single pass
- → good for small - medium dataset
- → Large dataset won't fit
- → $J(\beta) = \sum_{i=1}^{m}(y_i - \hat{y})^2$

Ordinary Least Square derivation (Loss fun)

→ OLS doesn't need any learning rate and gradient

$$J(\beta) = (Y - \hat{Y})^2 \Rightarrow (Y - X\beta)^2$$

$$\Rightarrow (Y - X\beta).(Y - X\beta) \Rightarrow \underbrace{Y^TY}_{0} - \underbrace{2\beta^TX^Ty}_{-2X^Ty} + \underbrace{\beta^TX^TX\beta}_{2X^TX\beta}$$

$$\frac{\partial J}{\partial \beta} = 0 - 2X^Ty + 2X^TX\beta$$

$\underbrace{\quad}_{0}$

$$\not{2}X^TX\beta = \not{2}X^Ty$$

$$\beta = \frac{X^Ty}{X^TX}$$

$$\Rightarrow (X^Ty)(X^TX)^{-1}$$

$\star\star$

$$J = y^2 - 2X\beta y + X\beta^2$$

$$\frac{\partial J}{\partial \beta} = 0 - 2X^Ty + 2X^TX$$

⇒ we have all parameters with us so no need of iterations but if there are multiple features then expensive on RAM and Transpose, inver

G.D (for ref):

$$\beta_{new} = \beta_{old} - \alpha \cdot \frac{X^T(X\beta - Y)}{n}$$

⇒ needs to run multiple iterations to reduce error

Numerical → Scaling
Categorical → Encoding

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
a = Scaler()
train_scaled = a.fit_transform(train)
```

fit → learn min, max
transform → apply

```
test_scaled = a.transform(test)
```

as a is already learned so just transform

```
O = OHE()
train_scaled_cat = O.fit_transform(train[cat])
test_scaled_cat = O.transform(test[cat])
```

13/06, ~~#106~~

~~Regularization~~ — ~~L1 (Lasso) & L2 (Ridge)~~

Feature	Type	Scaling (ML 6)
X_1	N	→ Standard scaling
X_2	N	
X_3	N	→ Min Max Scaling
X_4	C	
X_5	N	→ One hot encoding
X_6	C	
X_7	N	→ Log scaling
X_8	C	→ Target encoding

Standard scaling:
```
obj1 = SC(X1, X2)
Train_S = obj1.fit_tran
sform(train)
Test_S = obj1.transform(test)
```
} 3 line of code

Min Max Scaling } 3 line of code

One hot encoding } 3 line of code

Log scaling } "

Target encoding } "

Numerical:
- Standard
- Min-max
- log scaling

Cat:
- One Hot Encod
- Target encodi

Column Transformer ⇒ Key : Value pair

$[X_1, X_2] \rightarrow a$

$[X_3, X_5] \rightarrow b$

$[X_4, X_6] \rightarrow c$

$[X_7] \rightarrow d$

$[X_8] \rightarrow e$

Scaling objects:
```
sc = st_sc()
m = min_max()
ohe = ohe()
l_sc = log_scaling()
tar_enc = target()
```

preprocessor = {
'stn', sc, a
'minmax', m, b
'ohe', ohe, c
'l_sc', l_sc, d
'tar', tar_enc, e}

```
preprocessor = ColumTransformer(
    transformers = [
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)],
    remainder = "passthrough")
X_train_processed = preprocessor.fit_transform(X_train_split)
X_test_processed = preprocessor.transform(X_test_split)
```

passthrough ⇒ any columns that are not mentioned
Kepp them in the output as-is. (no transform

TRAIN MODEL :-

```
sgd_reg = SGDRegressor(max_iter=1000, eta0=0.01, random_state=5)
sgd_reg.fit(X_train_processed, y_train_split)
```

`sgd_reg.coef_` `sgd_reg.intercept_`

```
y_test_pred = sgd_reg.predict(X_test_processed)
y_train_pred = sgd_reg.predict(X_train_processed)
mse_train = mean_squared_error(y_train_split, y_train_pred)
mse_test = mean_squared_error(y_test_split, y_test_pred)
```

OLS

```
linear_reg_ols = LinearRegression()
```

Train: `linear_reg_ols.fit(x_train_processed, y_train_split)`

```
y_test_pred = linear_reg_ols.predict(x_test_processed)
y_train_pred = linear_reg_ols.predict(x_train_processed)
mse_train = mean_squared_error(y_train_split, y_train_pred)
mse_test = mean_squared_error(y_test_split, y_test_pred)
```

Save Artifacts

```
import joblib
model_filename = "artifacts/sgd_reg_model.joblib"
preprocessor_filename = "artifacts/data_preprocessor.joblib"
joblib.dump(sgd_reg, model_filename)
joblib.dump(preprocessor, preprocessor_filename)
```

Load model file

```
sgd_loaded_model = joblib.load(model_filename)
sgd_loaded_preprocessor = joblib.load(prepro_filename)
sgd_loaded_model.predict(input data)
```

scikit-learn → joblib → scientific
python modules → pickle → python code
pytorch → .pt → deep Learni

Regularization

→ How can we make sure our model coefficients are small alongside minimum error. Ex: m_1, m_2, m_3, m_4 → try use all coefficients instead of growing value of one

→ Also to prevents overfitting

→ Improve generalization

→ Regularization introduces errors / penalty to generate a generalized model

Hyper parameter Tuning :- Grid Search CV

```
Learning rate = [0.1, 0.01, 0.2, 0.5]
Optimizer = [OLS, SGD]
batch Size = [1, 10, 50, 100, N]
regularization_λ = [0.1, 0.2, 0.3, 0.8, 1]
```

L1 : LASSO Regularization

$loss_{new}$ = Modified MSE + $\lambda \cdot \Sigma |\beta_i|$

⇓

$400 + \lambda \cdot 4003$

β	
m_1	4000
m_2	1
m_3	0
m_4	2

$\Rightarrow \underline{4003}$

less error when $400 + \lambda \cdot 10$

m_1	0
m_2	10
m_3	0
m_4	0

$$Loss_{new} = \frac{1}{2n}\sum_{i=1}^{n}(y-\hat{y})^2 + \lambda \cdot \Sigma |\beta_i|$$

↳ Derivative

$$\Rightarrow \underbrace{\frac{1}{n}X^T(\hat{y}-y)}_{\text{Error term}} + \underbrace{\lambda \cdot Sign(\beta_i)}_{\text{L1 penalty term}}$$

14/06

if $\beta > 0 \rightarrow +1$

$\beta < 0 \rightarrow -1$

$\beta == 0 \rightarrow$ undefined

Scaling

$$\frac{X - X_{min}}{X_{max} - X_{min}}$$

L2 : RIDGE Regularization

$Loss_{new}$ = Modified MSE + $\lambda \cdot \Sigma \beta_i^2$

↳ Derivative = $\underbrace{\frac{1}{n}X^T(\hat{y}-y)}_{\text{Error Term}} + \underbrace{2 \cdot \lambda \cdot \beta}_{\text{L2 penalty term}}$

→ L2 keeps multico-linearity, where as L1 drops colinear columns

→ L1 will be used for simple and small model

Elastic Net (L1_ratio = 0.3)

L1 — 30% penalty

L2 — 70% comes from L2

$Loss_{new}$ = Modified MSE + $\underbrace{\lambda \cdot \Sigma |\beta_i|}_{30\%} + \underbrace{\lambda \cdot \Sigma \beta_i^2}_{70\%}$

→ ElasticNet is best of L1 and L2

```
from sklearn.linear_model import Ridge, Lasso
```

Lasso Regression

```
lasso = Lasso(alpha=0.1, random_state=42, max_iter=1000)
lasso.fit(X_train_processed, y_train_split)

y_train_pred_lasso = lasso.predict(X_train_processed)
y_test_pred_lasso = lasso.predict(X_test_processed)

lasso_rmse = np.sqrt(mean_squared_error(y_train_split, y_train_pred
                                        _lasso))
```

Replace with Ridge for other regularization

Regularization:
80% data only considers as correct

OHE : One Hot Encoding

Colours:

B → OHE → 100 ; G → OHE → 010 ; R → OHE → 001

P → OHE → 000
V → OHE → 000
⇒ unknown

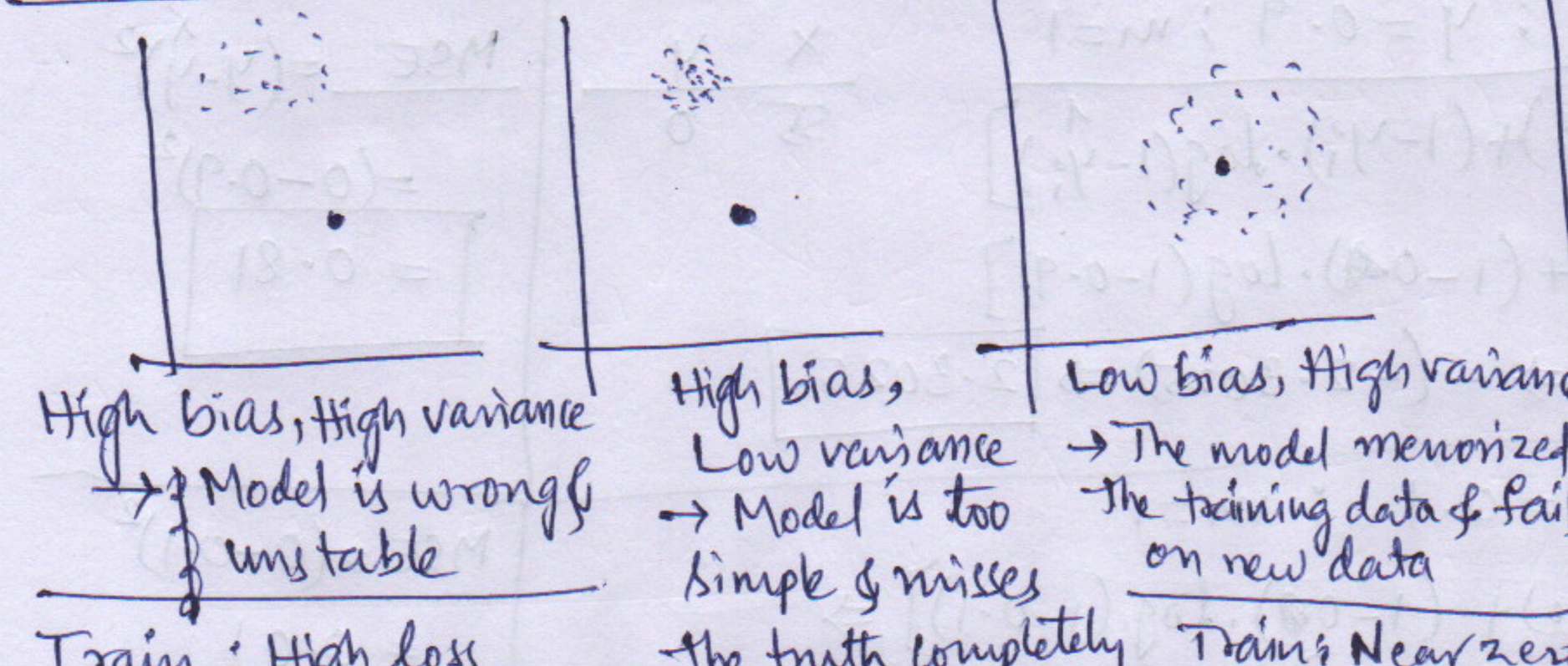
handle_unknown = 'ignore',
drop = 'first'

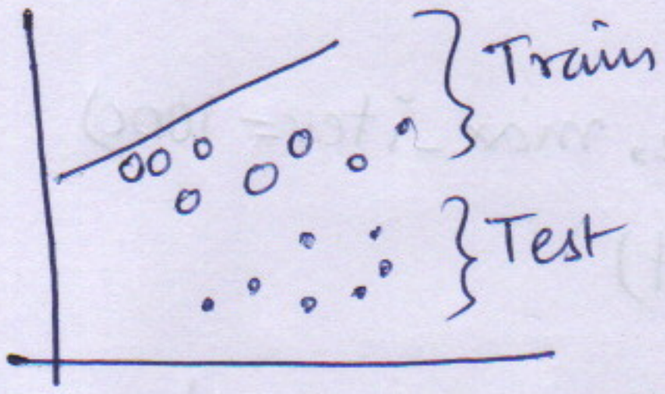
MLS
17/06

Bias & Variance

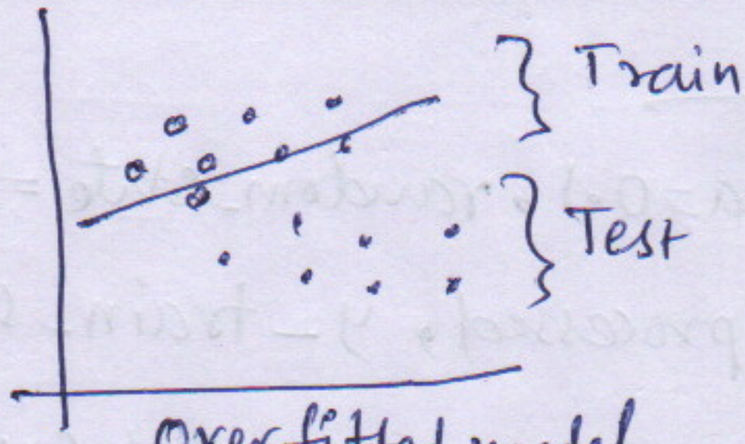
Bias : is under fitting ; variance : is overfitting model

Ideal : low bias model ; ideal model : Low variance

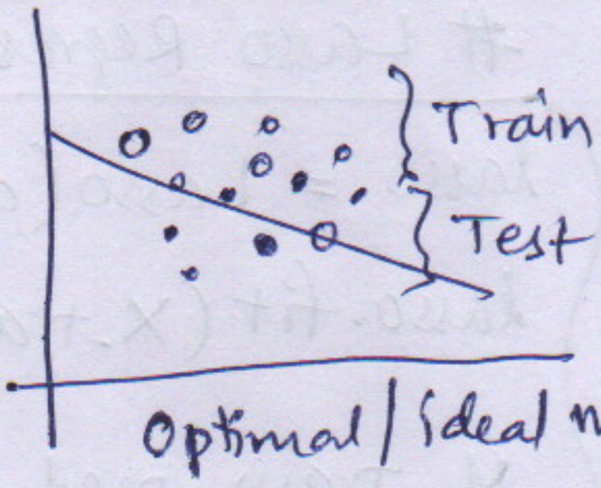




Under fitting model



Over fitted model



Optimal / Ideal model

Bias → fit of train data

Variance → Test performance

Low bias → good fit on train data

Low Variance → good fit on test data

Logistic Regression Algorithm [classification]

Logistic → Activation function (probability conversion)

Regression → $y = mx + c$

→ Linear regression + Activation function.

$z = mx + c \Rightarrow$ Sigmod (z) → 0.7 → 70%, 0.4 → 40%, 0.2 → 20% probabilities

Log Loss function in Logistic Regression:

$$J(\beta) = -\frac{1}{N}\sum_{i=1}^{n}\left(y_i \cdot \log(\hat{y}_i) + (1-y_i)\cdot \log(1-\hat{y}_i)\right)$$

1) Imagine: $y = 0 \; ; \; \hat{y} = 0.9 \; ; \; m = 1$

$$\Rightarrow -\left[y_i \cdot \log(y_i) + (1-y_i)\cdot \log(1-\hat{y}_i)\right]$$

$$\Rightarrow -\left[0 \cdot \log(0) + (1-0)\cdot \log(1-0.9)\right]$$

$$\Rightarrow -\log(0.1) \Rightarrow -(-2.3025) \Rightarrow \boxed{2.3025}$$

2) $y = 0 \; ; \; \hat{y} = 0.1 \; ; \; m = 1$

$$\Rightarrow -\left[0 \cdot \log(0) + (1-0)\cdot \log(1-0.1)\right] \Rightarrow$$

$$\Rightarrow -\left[1 \cdot \log(0.9)\right] \Rightarrow 0.045$$

X	y
55	0

$MSE = (y - \hat{y})^2$

$= (0 - 0.9)^2$

$= 0.81$

$MSE = (0 - 0.1)^2$

$= 0.01$

$MSE = \frac{1}{n}\sum(y - \hat{y})^2$

derivation of MSE gives gradient → intern direction & Magnitude

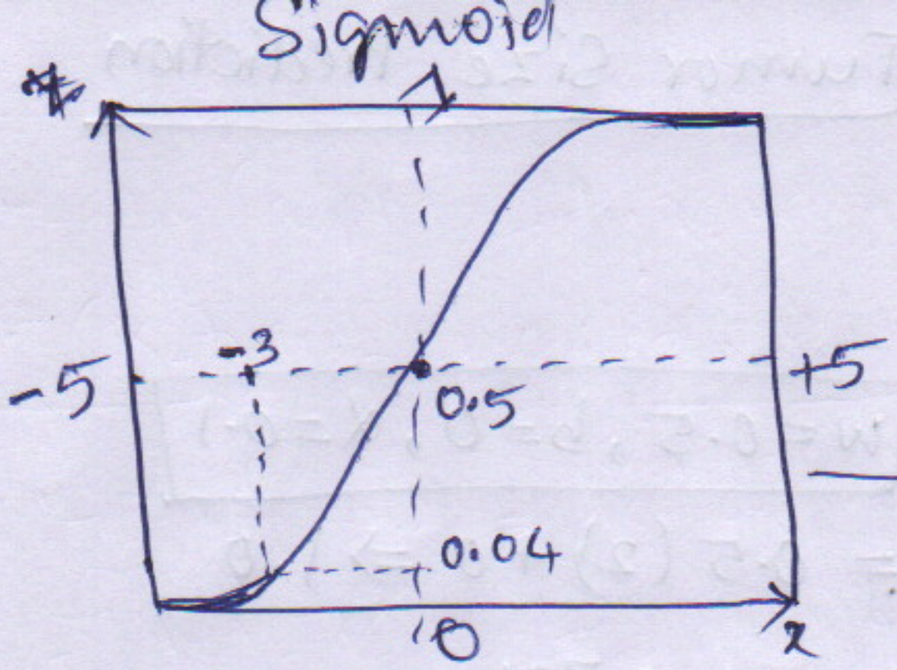
Sick

> 60% → sick

Case	y	y_pred	New_loss	MSE
Case-1	0	0.9	2.3025	0.81
Case-2	0	0.1	0.045	0.01

Remarks: when the error is high loss is high

when the error is low loss is low in



$-\infty$ to $+\infty$

$0 \rightarrow 0.5$

$-5 \rightarrow 0.066$ [close to 0]

$+5 \rightarrow 0.99$ [close to 1]

y	$\hat{y}$	$\hat{y} > 0.5$
1	0.9	1
0	0.2	0
1	0.8	1
0	0.3	0

Sigmoid → +1, 0

Tanh → +1, −1

Relu → max, 0

ML9

20/06

Logistic Regression

(1) Linear equation: $z = wx + b$

(2) Sigmoid prediction: $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$

(3) Log loss function(J): $-[y \cdot \log(1-\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$

Gradient: derivative of Loss func

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b}$$

1) $\frac{\partial J}{\partial \hat{y}} = -[y \cdot \log(1-\hat{y}) + (1-y)\log(1-\hat{y})]$

$\frac{\partial J}{\partial \hat{y}} \Rightarrow -\left[y \cdot \frac{1}{\hat{y}} + (1-y) \cdot \frac{-1}{1-\hat{y}}\right] \Rightarrow -\left(\frac{y}{\hat{y}} - \frac{(1-y)}{(1-\hat{y})}\right)$

$\Rightarrow -\left[\frac{y(1-\hat{y}) - \hat{y}(1-y)}{\hat{y}(1-\hat{y})}\right] \Rightarrow -\left[\frac{y - y\hat{y} - \hat{y} + y\hat{y}}{\hat{y}(1-\hat{y})}\right]$

$$\boxed{\frac{\partial J}{\partial \hat{y}} \Rightarrow \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}}$$

2) $\frac{\partial \hat{y}}{\partial z} = \hat{y} \cdot (1-\hat{y})$

3) $\frac{\partial z}{\partial w} = x$ | $\frac{\partial z}{\partial b} = 1$

Slope:

$$\frac{\partial J}{\partial w} = \left[\frac{\hat{y} - y}{\hat{y}(1-\hat{y})}\right]\left[\hat{y}(1-\hat{y})\right][x]$$

Co-efficient:

$$\frac{\partial J}{\partial b} = \left[\frac{\hat{y} - y}{\hat{y}(1-\hat{y})}\right]\left[\hat{y}(1-\hat{y})\right][1]$$

$$w_{new} = w_{old} - \alpha \cdot \text{gradient} \rightarrow \text{w.r.t } w$$

$$b_{new} = b_{old} - \alpha \cdot \text{gradient} \rightarrow \text{w.r.t } b$$

Logistic Regression : Tumor Size Prediction

Tumor Size(X)	Malignant(y)
2	1

→ Initialize random w, b : $w=0.5, b=0, \alpha=0.1$

Iteration - 1

1) Calculate $(z) = w \cdot x + b = 0.5(2) + 0 \Rightarrow 1.0$

2) Sigmoid $(\sigma) = \frac{1}{1+e^{-2}} = \frac{1}{1+e^{-1}} \Rightarrow 0.731$

Current State : The model predicts 73.1% chance of Tumor

3) Calculate Gradient $(J) = \frac{\partial J}{\partial w} = (\hat{y}-y) \cdot x \Rightarrow (0.731-1) \cdot 2$
$\Rightarrow -0.538$

$\frac{\partial J}{\partial b} = (\hat{y}-y) \cdot 1 \Rightarrow -0.269$

4) Update w & b :

$w_{new} = w_{old} - (\alpha \cdot Gradient_w) \Rightarrow 0.5 - (0.1(-0.538)) \Rightarrow 0.5538$

$b_{new} = b_{old} - (\alpha \cdot Gradient_b) \Rightarrow 0 - (0.1 - (-0.269)) \Rightarrow 0.0269$

Iteration - 2

1) $z = 0.5538 \cdot 2 + 0.0269 \Rightarrow 1.1345$

2) $\sigma \Rightarrow \approx 0.7567$

3) prediction moved from 0.731 to 0.756 which is closer to 1

3) Gradient $(J) = \frac{\partial J}{\partial w} = (\hat{y}-y) \cdot x \Rightarrow (0.756-1) \cdot 2 \Rightarrow -0.488$

$\frac{\partial J}{\partial b} = (\hat{y}-y) \cdot 1 \Rightarrow (0.756-1) \Rightarrow -0.244$

4) $w_{new} = w_{old} - (\alpha \cdot Gradient_w) \Rightarrow 0.55 - (0.1(-0.488)) \Rightarrow 0.5988$

$b_{new} = b_{old} - (\alpha \cdot Gradient_b) \Rightarrow 0.0269 - (0.1 \cdot (-0.244)) \Rightarrow 0.0513$

Column Transformer

transformer.append("impute", numeric_cols)

Simple Imputer

missing_value = 0, strategy = "median"
+
StandardScaler()

Stratify : Divides the train and test in correct Ratio of the classes.

Transformer (Imputation + Scaling)

ColumnTransformer (transformer)

ROC Curve

TP : model predicts positive, actual is positive
TN : model predicts negative, actual is negative
FP : model predicts positive, actual is negative
FN : model predicts negative, actual is positive

TPR: True Positive Rate

How many actual positive, caught correctly $= \frac{TP}{TP+FN}$ (Recall)

FPR: False Positive Rate

How many negatives are wrongly marked as positive $= \frac{FP}{FP+TN}$

Identifying an email as Spam or Not: Logistic Regression

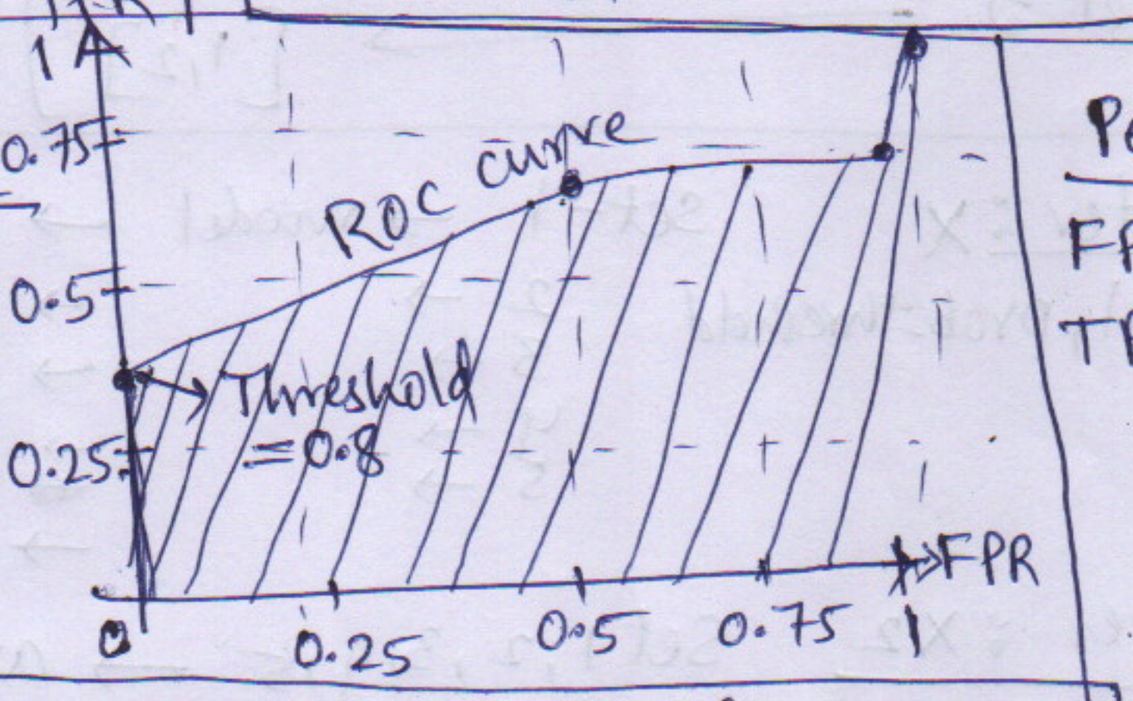
Email From	y	$\hat{y}$	Threshold (0.8)
A	1	0.95	1
B	0	0.7	0
C	1	0.6	0
D	0	0.4	0
E	1	0.2	0

TP → 1 (A) TN → 2 (B, D)
FP → 0 FN → 2 (C, E)

$$TPR = \frac{TP}{TP+FN} = \frac{1}{1+2} = 0.33$$

$$FPR = \frac{FP}{FP+TN} = \frac{0}{0+2} = \infty$$

Threshold	TPR	FPR	dist corner
0	1.0	1.0	0
0.4	0.67	1.0	0.33
0.6	0.67	0.5	0.35
0.8	0.33	0	0.67
1.0	0	0	0

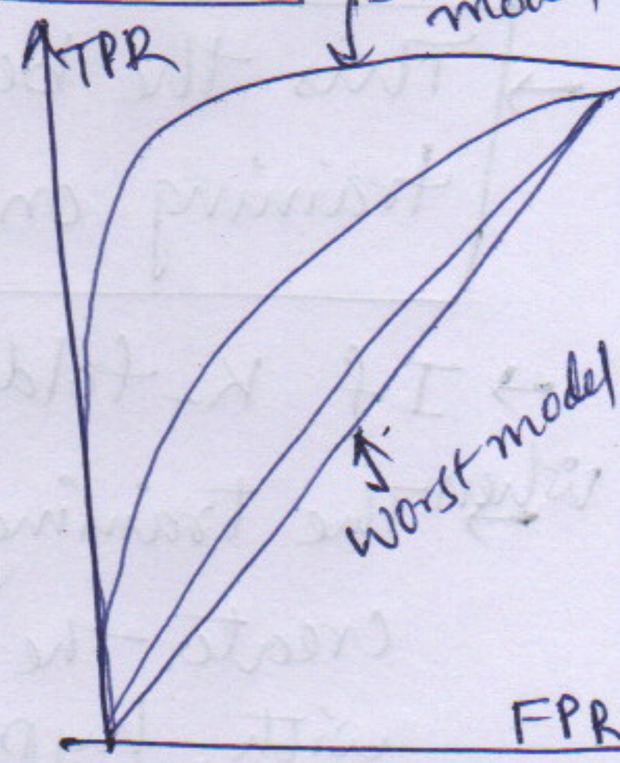


Perfect Model
FPR = 0
TPR = 1

ROC curve : TPR vs FPR for all thresholds

AUC: Area Under Curve

→ Higher the area better the model (numerical val)
→ AUC = 1 → perfect model



Find heighest distance of TPR and FPR

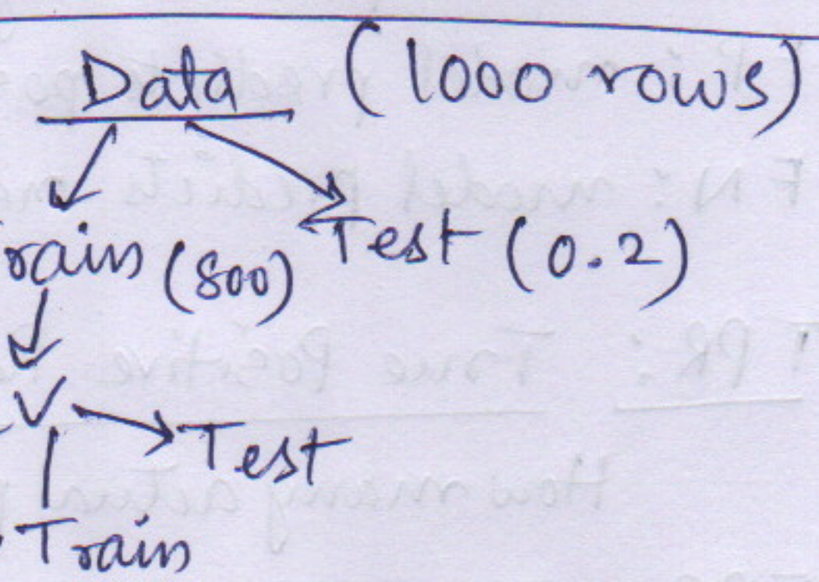
1) Youden's J statistic — $y = TPR - FPR$ → Hight distance

2) Distance to corner — $d = \sqrt{(1-TPR)^2 + (0-FPR)^2}$ → min distance

---

CROSS VALIDATION (CV)

→ Instead of one fixed data spht, we train multiple times, each with different set



~~K-fold~~

K-fold = ~~5~~ | Creates 5 set of data

test ratio = $\frac{1}{K\text{-fold}} \Rightarrow \frac{1}{5} = 20\% \Rightarrow 800 \times 20\%$

Test ⇒ 160

Train = 800 − 160 = 640

Example:

Data = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

K-fold = 5	Train	Test
Set 1 =	[1, 2, 3, 4, 5, 6, 7, 8]	[9, 10]
Set 2 →		[7, 8]
Set 3 →		[5, 6]
Set 4 →		[3, 4]
Set 5 →		[1, 2]

~~Test~~ data in Test will be used only once

→ Each data becomes test data once

Parameter : X — $\alpha = 0.01$, prob-threshold

Set-1 → model → performance → 96%

2 → → 84%

3 → → 60%

4 → → 80%

5 → → 70%

average gives the model performance

Parameter : X2 — $\alpha = 0.02$, prob-threshold — Set 1, 2, 3, 4, 5 → average of performance

→ This the best example of model selection ~~and~~ on model training on different hyper parameters

→ If K-folder higher the computation will be more

When the training meets satisfield performance → then we will create the final model on Train data. Else we continue with hyperparameter and finally select the best model.

KNN → K Nearest Neighbor (Supervised algorithm)

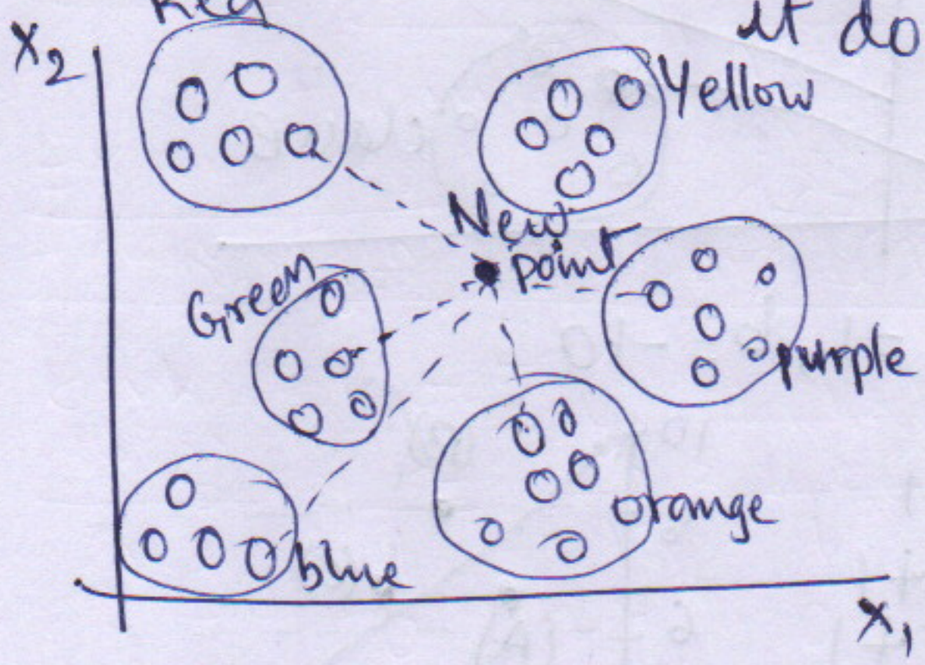
K = Numbes of neighors

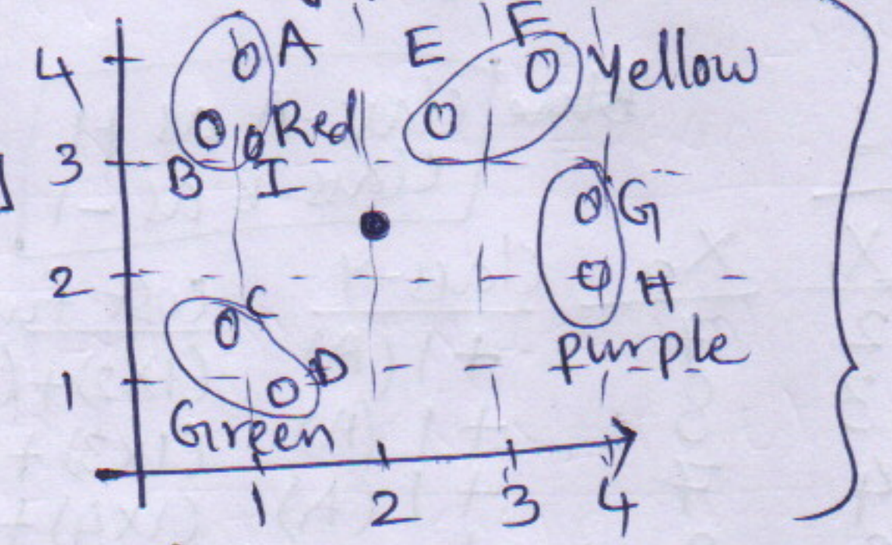
→ used for
1) Classification
2) Regression.

→ No mathematical equation
→ No Lines
→ No curves

→ Used when,
data doesn't for a nice Line

→ Instead of relying on equation, rely on data to make decission
→ We don't compute any parameters (α, β, lr)
→ It's a lazy learner algorithm because it doesn't learn any parameters.





Classificatio

Distance Formula :-

1) Euclidean distance: $\sqrt{\sum (x_i - x_j)^2}$
2) Manhatton distance: $\sum |x_i - x_j|$

N → A (2,2) (4,4)
N → B (1,3)
N → C (1,2)
N → H (4,2)
N → I (1,1)

distance (Euclidean)

$\sqrt{(2-4)^2 + (2-4)^2} = 2.23$
$\sqrt{(2-1)^2 + (2-3)^2} = 1.414$
$\sqrt{(2-1)^2 + (2-2)^2} = 1$ ★
$\sqrt{(2-4)^2 + (2-2)^2} = 2$
$\sqrt{(2-1)^2 + (2-1)^2} = 1.414$

Sort

→ Sort distance calculate

dist	Par
1	C
1.414	B
1.414	I
2	H
2.23	A

K = 3
K → select no of neighbors
→ choose odd numbers

N → (C, B, I)
Green Red Red

→ How to decide K value
Elbow method, WCSS

★ K = 1 → Preceision, Recall
K = 2 → " "
K = 3 → " "

→ New point (N) will be classified as RED

For Regression : Y - Salary

N → (C, B, I) → Salary
Salary 100K, Salary 100K, 100K

average = 400K $\left(\frac{C+B+I}{3}\right)$

N = 400K based on (B, I)

MLA2
28/06

SVM: Support Vector Machines [Supervised Algorithm]

→ Used for classification & Regression.

2 types:

Type-1: Primal form (Linear SVM) → w, b (learning params) Equation of Line

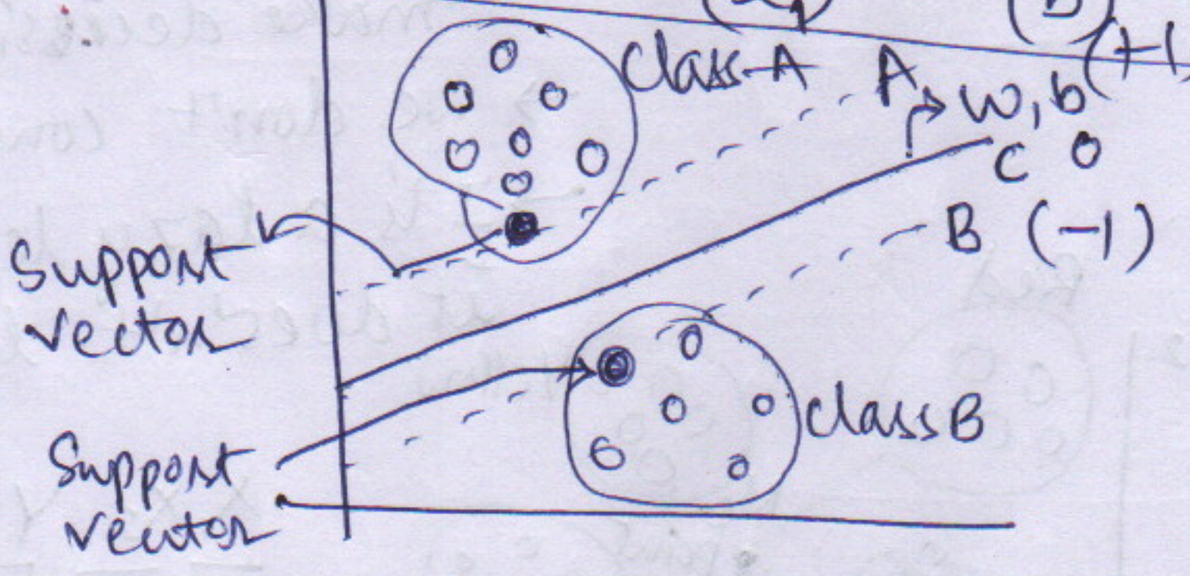
Type-2: Dual form (Kernalized SVM) → Each Datapoints → Learn weightage + bias (α_i) (b)

Classification :-

$A \rightarrow wx + b = +1$

$B \rightarrow wx + b = -1$

$C \rightarrow wx + b = 0$

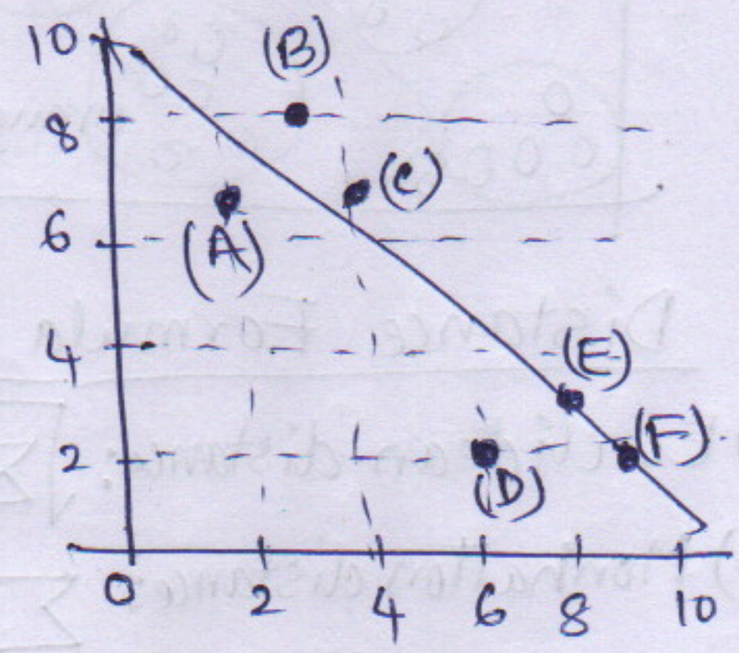


Blue | Class-A as +1, Class-B as -1

$w_1 = 1$; $w_2 = 1$, $b = -10$

Dataset:

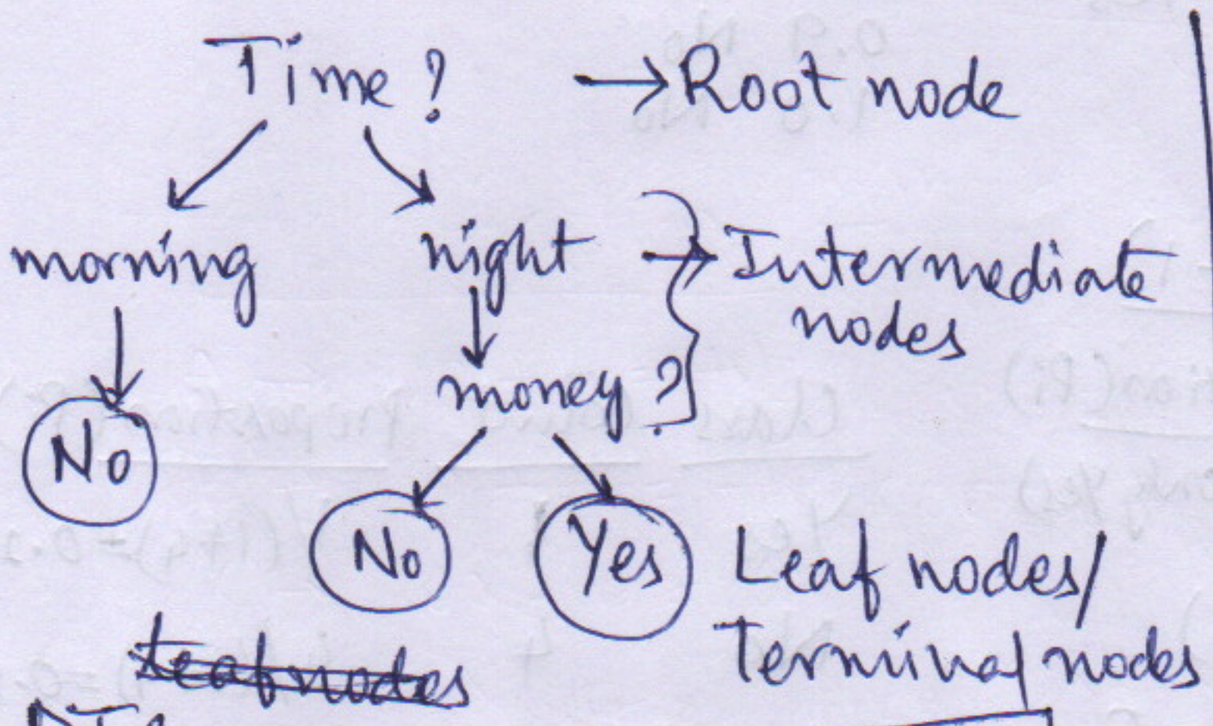
Point	X_1	X_2	Class-Y	cost fun
A	2	7	+1 (A)	$(1\times2)+(7\times1)-10=-1$
B	3	8	+1 (A)	$(1\times3)+(8\times1)-10=+1$
C	4	7	+1 (A)	$(1\times4)+(1\times7)-10=+1$
D	6	2	-1 (B)	$(1\times6)+(1\times2)-10=-2$
E	7	3	-1 (B)	$(1\times7)+(1\times3)-10=0$
F	8	2	-1 (B)	$(1\times8)+(1\times2)-10=0$



Decision Tree - Supervised algo

→ DT breaks the data into smaller subsets by asking a series of targetted, conditional "if-then-else" questions.

Intution: How Humans take decision - Go to night club



Time (X_1)	Money (X_2)	Decision (Y)
Morning	Yes	No
Morning	No	No
Night	No	No
Night	Yes	Yes

DT:
If-then-else decisions navigated via conditional control statement

→ Leaf nodes → represents model output / predictions

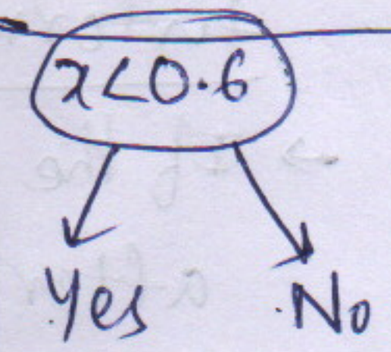
Decision Tree Classifier

→ Purity - Impurity
- → Entropy
  - → Information Gain
  - IG(parent) — IG(child)
- → Gini Impurity
  - → Gini Gain
  - GG(parent) — GG(child)

Impurity

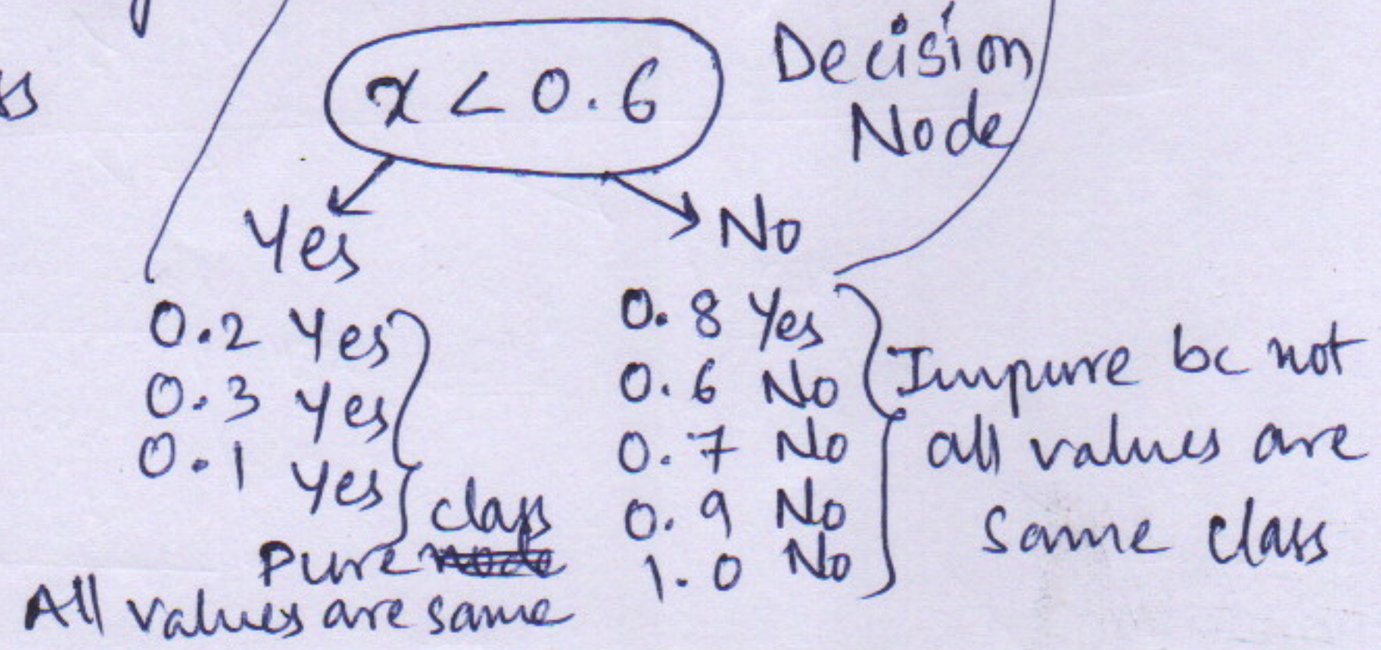
x	y
0.2	Yes
0.3	Yes
0.1	Yes
0.8	Yes
0.6	No
0.7	No
0.9	No
1.0	No

if $x < 0.6$
Yes
else
No



Pure Node - all samples belong to same class

Impure Node - Mixed class



1) Entropy is the measurement of uncertainity

↳ $-\sum_{i=1}^{n} P_i \log_2(P_i)$

2) Gini Impurity

↳ $1 - \sum_{i=1}^{n} P_i^2$

$x < 0.6$

Yes ↙ ↘ No

Yes	No
0.2 Yes	0.8 Yes
0.3 Yes	0.6 No
0.1 Yes	0.7 No
	0.9 No
	1.0 No

Entropy (0-1)

Class	Count	Proportion (P_i)
Yes	3	1 (only Yes)

$= -\sum P_i \log_2(P_i)$

$= -(1. \log_2(1)) = 0$

Impurity = 0

Class	Count	Proportion (P_i)
Yes	1	$1/(1+4) = 0.2$
No	4	$4/(1+4) = 0.8$

$= -\sum P_i \log_2(P_i)$

$= -(0.2 \log_2 0.2 + 0.8 \log_2 0.8)$

Impurity = 0.722

Worst Condition of DT

Class	Count	Proportion
Yes	4	0.5
No	4	0.5

$= -(0.5 \log_2 0.5 + 0.5 \log_2 0.5)$

Impurity = 1

Information Gain : we will use on entropy

→ If the split is good, IG calculates how much entropy reduces after a split.

→ Helps us to choose feature (least impurity) that gives highest IG.

$$\text{Information Gain} = \text{Entropy}_{parent} - \sum \frac{n_{child}}{n_{total}} \times \text{Entropy}_{child}$$

Step-1 : Calculate parent entropy (before split)

(X_1) weather	(X_2) Temp	(Y) play-Tennis
Sunny	Hot	No
Sunny	Mild	No
Cloudy	Hot	Yes
Rain	Mild	Yes
Rain	Cool	Yes
Cloudy	cool	Yes
Rain	Hot	No
Sunny	Cool	Yes

Class	Count	Proportionate
Yes	5	5/8 = 0.625
No	3	3/8 = 0.375

$$\text{Entropy (Parent)} = -(0.625 \cdot \log_2 0.625 + 0.375 \cdot \log_2 0.375) = 0.954$$

Step-2 : Try each feature

Weather (X_1)
- → Calc Entropy (X_1)
- → IG (Parent-child)

Temp (X_2)
- → Calc Entropy (X_2)
- → IG (parent-child)

Whichever will be having higher IG becomes the root node

Feature 1 : weather (X_1)

weather	Count	Yes	No	Entropy
Sunny	3	1	2	$-(\frac{1}{3}\log_2\frac{1}{3}+\frac{2}{3}\log_2\frac{2}{3}) = 0.918$
Cloudy	2	2	0	$-(\frac{2}{2}\log_2\frac{2}{2}+0) = 0$
Rain	3	1	1	$-(\frac{1}{2}\log_2\frac{1}{2}+\frac{1}{2}\log_2\frac{1}{2}) = 0.918$

weighted avg entropy after split:

$$\frac{3}{8}\times 0.918 + \frac{2}{8}\times 0 + \frac{3}{8}\times 0.918 = 0.688$$ (entropy/Impurity)

Information Gain = Entropy Parent − New Entropy

$= 0.954 - 0.688$

IG = 0.266

Feature 2 : Temp (X_2)

Temp	Count	Yes	No	Entropy
Hot	3	1	2	$-(\frac{1}{3}\log_2\frac{1}{3}+\frac{2}{3}\log_2\frac{2}{3}) = 0.9$
Mild	2	1	1	$-(\frac{1}{2}\log_2\frac{1}{2}+\frac{1}{2}\log_2\frac{1}{2}) =$
Cool	3	3	0	$-(\frac{3}{3}\log_2\frac{3}{3}+0) =$

weighted avg entropy after split:

$$\frac{3}{8}\times 0.918 + \frac{2}{8}\times 1 + \frac{3}{8}\times 0 = 0.594$$

$=$

$IG = \text{Entropy}_{parent} - \text{New Entropy}$

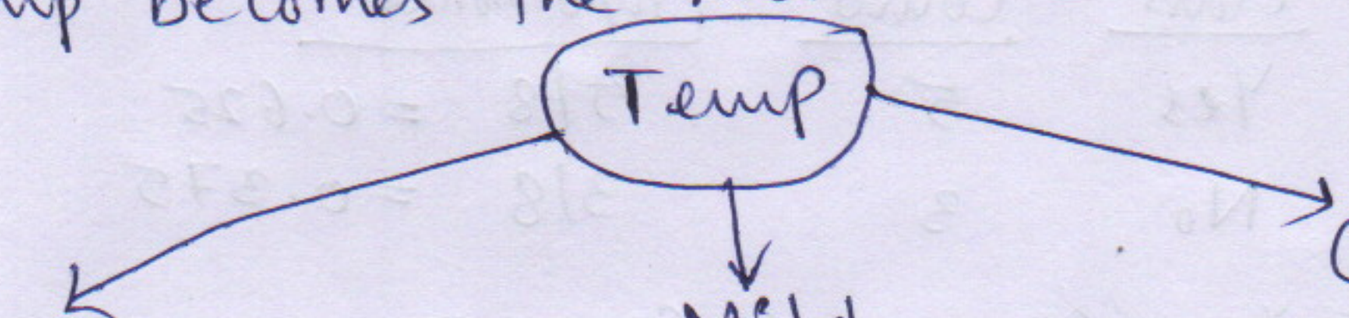
$= 0.954 - 0.594$

IG = 0.368

IG (weather) IG (Temp)

0.266 < 0.360 *

Temp becomes the root node as its IG is higher



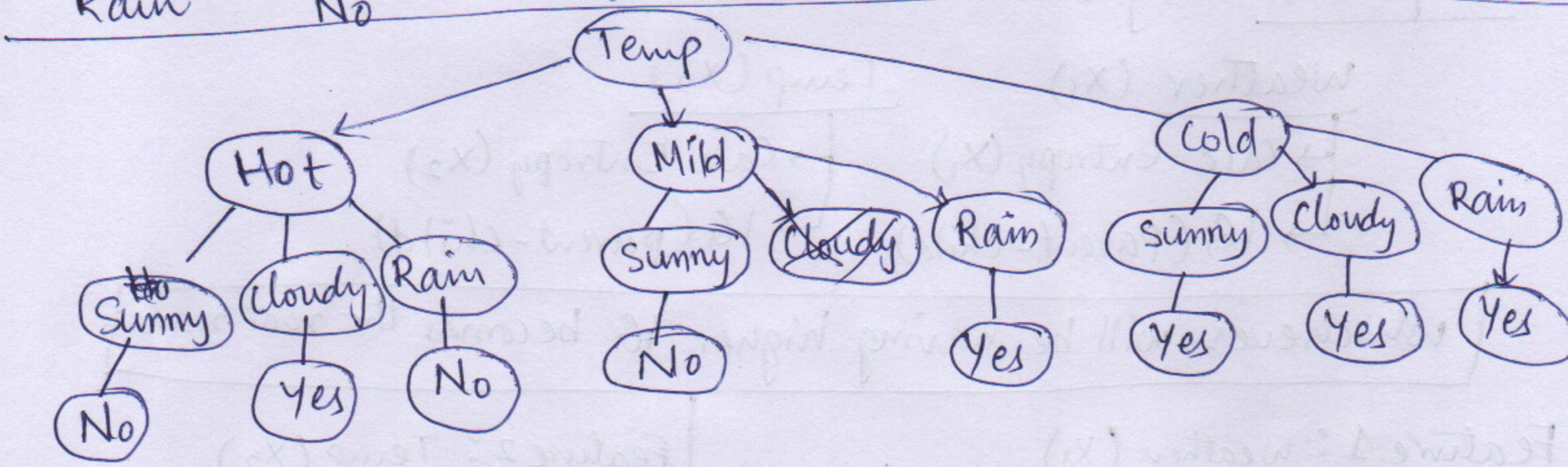
Hot

weather (X)	Y
Sunny	No
Cloudy	Yes
Rain	No

Mild

weather (X)	Y
Sunny	No
Cloudy	—
Rain	Yes

weather (X)	Y
Sunny	Yes
Cloudy	Yes
Rain	Yes



MLLB 12/07

Decision Tree (DT) – Regressor – Supervised algo

Dataset:

ID	Sq.Feet	Price (L)
1	800	50
2	850	52
3	900	54
4	1500	90
5	1600	95
6	1700	100

Total: 441

Avg price = $\frac{441}{6} = 73.5$

μ (price) = 73.5

Step1: Compute parent variance (price)

$$\text{variance} = \frac{1}{n}\sum(y_i - \mu)^2$$

$$= \frac{1}{6}\left((50-73.5)^2+(52-73.5)^2+(54-73.5)^2 + (90-73.5)^2+(95-73.5)^2+(100-73.5)^2\right)$$

var = 471.9

Step-2: Select column i.e. Sq. Feet

→ Sort data → 800, 850, 900, 1500, 1600, 1700

→ Split point/(mid point 825, 875, 1200, 1550, 1650)

Starting Split at 1200:

Sq.Feet ≥ 1200

Left		Right	
800	50	1500	90
850	52	1600	95
900	54	1700	100
	156/3		285/3
μ =	52	μ =	95

$$var = \frac{1}{3}\left((50-52)^2+(52-52)^2+(54-52)^2\right)$$

$$var = 2.67$$

$$var = \frac{1}{3}\left((90-95)^2+(95-95)^2+(100-95)^2\right)$$

$$var = \frac{50}{3} = 16.67$$

weighted variance:

$$var\text{-}split(1200) = \frac{3}{6} \times 2.67 + \frac{3}{6} \times 16.67 = 9.67$$

variance reduction (1200) = 471.9 − 9.67

= 462.23

Similarly, we need to calculate variance reduction at 825, 875, 1550, 165

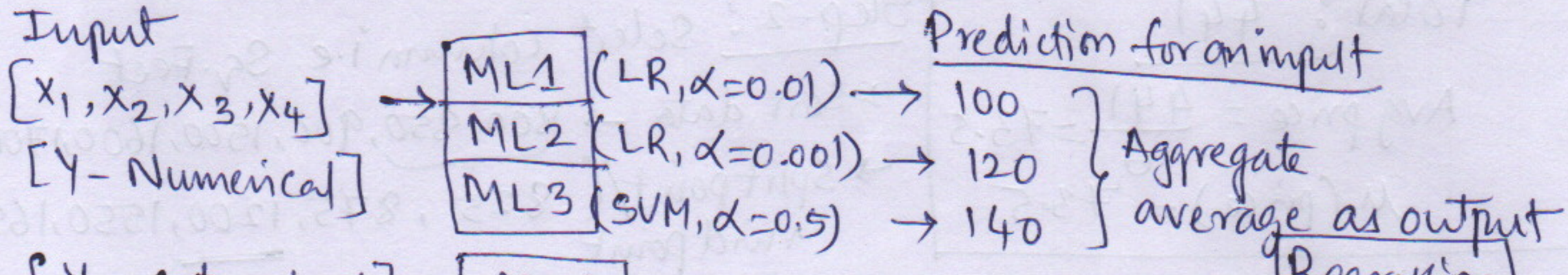
Lets, imagine VR at 1200 is highest, which becomes at **Root Node**

Ensemble method/learning/Technique

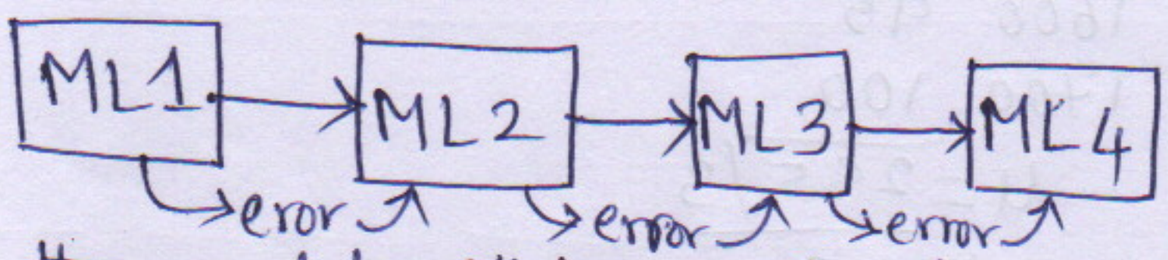
- Bagging → Random forest (Bagging + Decision Tree)
  - → Bootstrap
- Boosting → XGboot, Gradient boosting
  - → (boosting + Decision Tree + other)

Ensemble method is ~~used~~ creating multiple models for a single task

Bagging: Multiple models will be trained in parallel model



Boosting: Multiple models are trained in sequential model



→ All the models will have simple architecture/base-model

Bootstrap is used in the context of Dataset :-

→ It is a dataset technique which we will use in DT.

B.T = 2

↓

Dataset

- Subset-1 → 63.2% of original data + 36.8% data will be repeated randomly (duplicates from 63.2% of data)
- Subset-2 (used for training) → 63.2% original data + 36.8% random data (used for validation)

X	Y
1	200
2	300
3	400
4	500
5	600
6	700

BT_Dataset-1

X	Y
1	200
2	300
2	300
4	500
5	600
1	200

Total count = 6

↓

OOB Dataset {3, 400; 6, 700}

BT_Dataset-2

X	Y
1	200
2	300
3	400
5	600
3	400
6	700

total count = 6

↓

4 500

OOB

Random Forest : (Bagging)

Dataset ⇒ n_estimators = 100 → 100 Bootstrap data → BTD1 BTD2 ... BTD10
i.e 100 DT

⇒ max_features = 2
→ for every DT node it selects 2 features only

} Bagging + Bootstrap + Decision Tree

→ calculate Parent variance.

→ root → X_1 → Sort values → mid point → Calc variance

X_3 ← max_feature = 2 → Sort val → mid point → Calc variance

→ Find the best mid point using the highest variance reduction.

→ repeat above until n_estimators with random features selected.

Final prediction :-

DT → Model (F_N)

DT → $F_N - \hat{y}$

N = 100

$$\hat{y}_{RF}(x) = \frac{1}{N}\sum_{i=1}^{N} f_N(x)$$

Example, House price prediction
Sqft = 1200, bedrooms = 3, Age = 3

$f_1(x) = 97.5$
$f_2(x) = 93.5$
$f_3(x) = 90$

$\hat{y}_N(x) = \frac{1}{3}(97.5 + 93.5 + 90)$

RF prediction = 93.75

Gradient Boosting: Sequential models

→ Its a DT with simple models
→ stumps [1,3,6,8] → depth
→ we use loss function → $\frac{1}{n}\sum_{i=1}^{n}(y-\hat{y})^2$

Dataset:

ID	Size	Beds	price(y)	γ_1
1	800	2	120	-34
2	900	2	130	-24
3	1000	3	150	-4
4	1100	3	170	+16
5	1200	4	200	+46

Step-0 → init model (F_0)
F_0 = mean(y) → 154
→ Calc residuals (γ_1)
$y - F = y - F_0$

Step-1: mid point of features + DT

feature	Threshold	Left(A)	Right(B)	mean(A)	mean(B)	SSE
Size	1050	[1,2,3]	[4,5]	-20.66	31	916
Size	950	[1,2]	[3,4,5]	-29	19.33	1316
Size	1150	[1,2,3,4]	[5]	-11.5	46	1475
Size	850	[1]	[2,3,4,5]	-34	8.5	2675
Beds	2.5	[1,2]	[3,4,5]	-29	19.33	1316
Beds	3.5	[1,2,3,4]	[5]	-11.5	46	1475

SSE → Sum of Square Error ⇒ 916

Data (X)
↓
Model [F_0]
↓
Avg = 154

GB(Regression)

(4) loss = $\frac{1}{n}\Sigma(y-f(x))^2$
gradient (first derivative)
→ $g = \frac{\partial L}{\partial f} = F - y$
Changing direction to lower the loss →
$-g = y - F$

lowest